Sensor Hub

Open Source Weather Station

Build and Configuration

v 1.0

October 29, 2026

1. Abstract 4
2. Background 4
3. Hardware 5
3.1 Raspberry Pi Zero 2W 7
3.2 Reset Button 7
3.3 Status LED 8
3.4 Power LED 8
3.5 RTL_433 Radio Dongle 8
4. Software Architecture 8
4.1 Two separate, communicating programs 9
4.2 Use of Bonjour services 9
4.3 REST API 10
4.4 Reverse proxy service for enabling remote access from anywhere 10
4.5 [Optional] REST API transport modes security - Default is open 11
4.6 Browser Wi-Fi setup 12
4.7 iPhone Wi-Fi setup (requires iOS developer and XCode tools) 15
4.9 rtl_433 (open source) 20
5.0 Software Design and Operation 21
5.1 Data Structures 21
5.1.1 SENSORS Table 21
5.2 Hub and App Control 23
5.3 Interfaces and Thread Signaling 23
5.3.1 processsensordata (Goroutine) 23
5.3.2 restservice (Goroutine) 23
5.3.3 expiredsensors (Goroutine) 24
5.3.4 monitorwifi (Goroutine) 24
5.3.5 writeledstate (Goroutine) 24
6.0 Sensor Hub Software Build 25
6.1 Update Raspberry Pi OS 25
6.2 Install git and sensorhub, sensorhubcontrol repositories 25
6.3 Install Mosquitto 25
6.4 Test Mosquitto 26
6.5 Create and Configure Log Files and working directory 26
6.6 Create logrotate configuration file 26
6.7 Install sqlite3 26
6.8 Configure Bonjour advertising 26
6.9 Add sensorhub.service: 27
6.10 Configure profile for local hotspot 27
6.10.1 Delete that connection: 27
6.11 Install rtl_433 radio software (to /usr/bin) 28
6.12 Configure temp-sensor-rtl service to systemd from repository directory 29
6.13 Copy sensorhub and hubcontrol files 30

6.14 Enable systemd services 30
6.15 Configure profile for local hotspot 30
10.0 Copy executables and templates to /usr/local/bin/sensorhub 31
11.0 Start sensorhub Service Under systemd 31
11.1 Step 1 - Create a systemd service file to run sensorhub on boot up 31
11.2 Step 2 - Check log of running service 32
12.0 Remote Access via pinggy.io 32
12.1 Step 1: Go to the pinggy.io subscription 32
12.2 Step 2: Configure the new tunnel 33
12.3 Step 3: Save the configuration 33
12.4 Step 4: Copy the SSH command block 33
12.5 Step 5: Create pinggy_tunnel.sh script 34
12.6 Step 6: Create a Systemd Service 34
12.7 Step 7: Reload Systemd and enable the service 34
12.8 Step 8: Access the REST service like this 34
12.9 Troubleshooting 34
13.0 systemd commands 35
14.0 Log commands 35
15.0 Viewing sqlite3 database 35
16.0 Testing REST Service from another terminal 36

1. Abstract

The Sensor Hub is a device that detects, receives, and transmits AcuRite (and other brand 433 MHz sensors) temperature/humidity sensor data. It uses a self-hosted WiFi hotspot for initial WiFi configuration, then restarts with the WiFi connected to the user-selected SSID. An RTL_433 radio receiver is connected to a USB port with an external antenna to receive the sensor signals. The RTL_433 software publishes sensor data to a local MQTT message broker service. The sensor data is collected from the MQTT service in a sqlite3 database. A subroutine thread constantly reads this database and publishes the sensor data using a REST API on local port 5000. The API also provides a means for applications to update sensor records in the sqlite3 database with user-specified names, locations, and descriptions.

It runs on a Raspberry Pi 32-bit or 64-bit device. The Raspberry Pi must have a WiFi interface. The hardware design includes optional power and status LEDs as well as a reset button that can be used to reset the Sensor Hub for a different WiFi SSID.

A separately available iOS app (**Sensor-Hub-Xcode,** open source in github.com) provides user access to initially configure the Sensor Hub, name it, select from available sensors, and customize sensor names. The Sensor Hub provides an option to incorporate a paid reverse proxy service making the sensor data available from anywhere on the Internet. Without the reverse proxy service, the browser or iOS app can only access the sensor hub data when it is connected on the same WiFi network as the sensor hub. The iOS app is provided as open-source rather than as a supported app on the App Store. As an alternative tool, the REST API is exposed so that users can manage the Sensir Hub directly using **curl** or a browser-based application.

2. Background

Weather sensors are ubiquitous. Vendors like AcuRite, LaCrosse, Oregon Scientific, and Ambient sell weather stations and sensors. Each vendor's sensors tend to pair only with their proprietary weather stations. If you happen to have sensors from multiple vendors, you end up with several different weather stations. Sometimes a vendor drops support for a product and you have to replace your weather sensors with another brand. This can become expensive and tiresome.

Recently, one of the largest vendors, AcuRite, announced that they were replacing their free app for monitoring up to five sensors connected to their proprietary hub with a new app. The new app claims it provides users with improved service and options. However, the new app has less capability than the previous app unless you subscribe to their service. Subscribing to the service only provides support for their devices, so you would still be limited in what sensors you could use.

Sensor Hub is intended to replace these proprietary hubs and services with an open-source integrated set of hardware and software supporting multiple vendor products while giving the user the ability to configure the sensors for both local and remote access. The user controls the service and is not limited to a specific vendor. Multiple Sensor Hubs can be configured and displayed.

For example, you can have a Sensor Hub (home) with several temperature/humidity sensors for your house and a second Sensor Hub (rv) with temperature/humidity sensors in a recreational vehicle. You can monitor your home and RV using the same software tools.

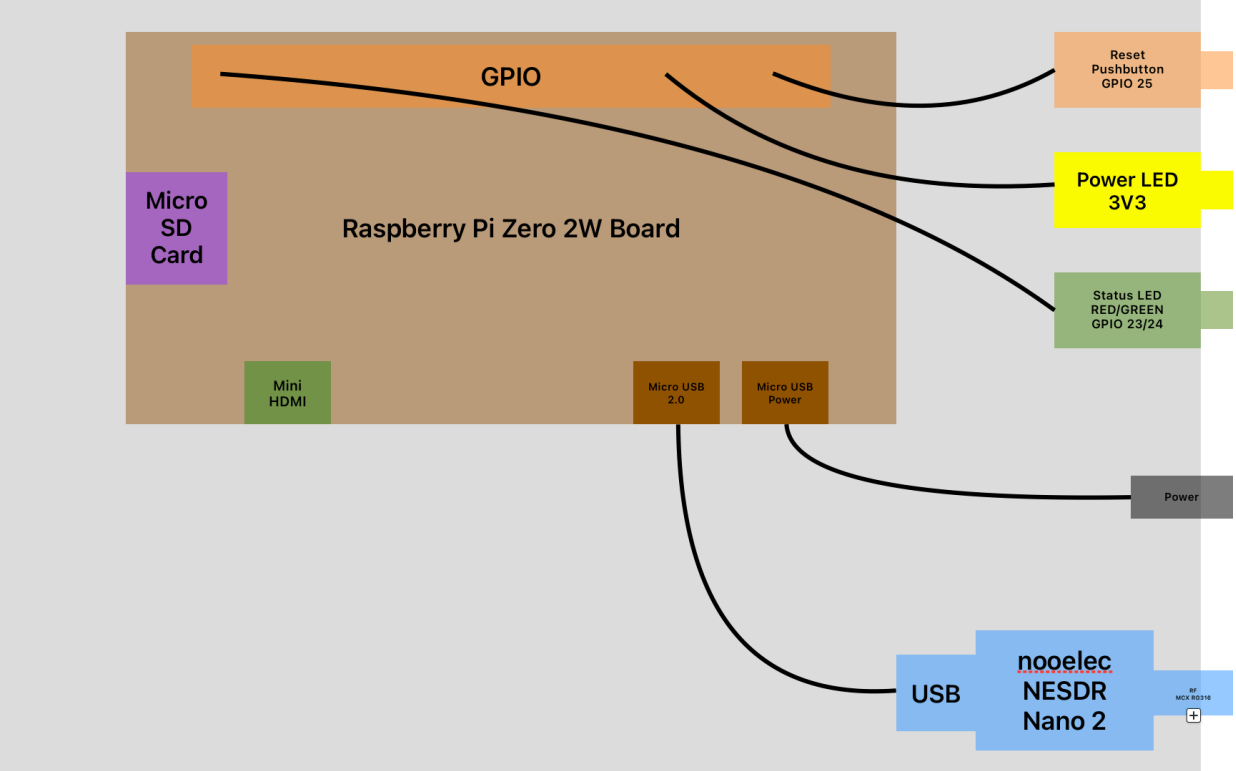
3. Hardware

A low-cost, readily-available hardware platform has been selected for the Sensor Hub. The minimum requirement is support for WiFi and 433MHz software defined radio. The Raspberry Pi family of CPU boards meets these needs. The basic PiZeroW is a single core 32-bit configuration that runs the latest Debian releases. For a few dollars more, the PiZero2W provides quad-core 64-bit functionality. Either processor is more than adequate to handle the Sensor Hub software requirements.

The block diagram shows the components of the Sensor Hub. One micro-USB port supplies power from a 2.0A, 5V wall adapter. The other micro-USB port serves as the interface for third-party RTL 433 SDR radios (e.g., nooelec NESDR Nano 2). Finally, the board's GPIO pins provide support for driving the reset button, power LED, and status LED. In our case, we use a multi-color red/green LED for status that is driven by two different GPIO pins.

Although **sensorhub** will run on a generic Raspberry Pi with just the RTL 433 USB dongle, the additional features of WiFi reset and status lights elminate the need to log into the Raspberry Pi and edit configuration files to move to a different WiFi network.

One important note: Be sure to use a high-quality USB-A to micro-USB cable between the RTL 433 radio dongle and the Pi. A bad cable will result in mysterious drop-outs between the SDR driver code and the USB bus.



Hardware block diagram

The sensor hub software has been tested on Raspberry Pi model Zero2W (64-bit) and model ZeroW (32-bit). It will also run on all newer versions of the Raspberry Pi.

3.1 Raspberry Pi Zero 2W

```
$ pinout
Description        : Raspberry Pi Zero2W rev 1.0
Revision           : 902120
SoC                : BCM2837
RAM                : 512MB
Storage            : MicroSD
USB ports          : 1 (of which 0 USB3)
Ethernet ports     : 0 (0Mbps max. speed)
Wi-fi              : True
Bluetooth          : True
Camera ports (CSI) : 1
Display ports (DSI): 0

,--ooooooooooooooooooo---.
|  1ooooooooooooooooooo J8|
---+     +---+  PiZero2W  c|
 sd|     |SoC|   Wi V1.0  s|
---+     +---+   Fi       i|
| hdmi            usb pwr |
`-|  |------------| |-| |-'


J8:
   3V3  (1) (2)  5V
 GPIO2  (3) (4)  5V
 GPIO3  (5) (6)  GND
 GPIO4  (7) (8)  GPIO14
   GND  (9) (10) GPIO15
GPIO17 (11) (12) GPIO18
GPIO27 (13) (14) GND
GPIO22 (15) (16) GPIO23
   3V3 (17) (18) GPIO24
GPIO10 (19) (20) GND
 GPIO9 (21) (22) GPIO25
GPIO11 (23) (24) GPIO8
   GND (25) (26) GPIO7
 GPIO0 (27) (28) GPIO1
 GPIO5 (29) (30) GND
 GPIO6 (31) (32) GPIO12
GPIO13 (33) (34) GND
GPIO19 (35) (36) GPIO16
GPIO26 (37) (38) GPIO20
   GND (39) (40) GPIO21
```

3.2 Reset Button

Press and hold this button for 5 seconds and it causes the sensor hub to enter configuration mode, broadcasting on its standalone WiFi hotspot. It will remain in this mode until power is cycled or an iOS app connects to configure it. The Status LED will be flashing red during this time.

Reset uses GPIO25, physical pin 18 of the 40-pin GPIO bus.

3.3 Status LED

The red/green status LED provides status indications for the sensor hub application. Red LED uses GPIO23, green LED uses GPIO24, physical pin 18 of the 40-pin GPIO bus.

- Solid Green - indicates Sensor Server is running, connected to the Internet, and OK
- Flashing Green - server is running and connected to WiFi, but has lost connection to the Internet.
- Solid Red - indicates Sensor Server has encountered an unrecoverable error. Power cycle to clear.
- Flashing Red - indicates Sensor Server is booting the application.

3.4 Power LED

The yellow LED lights when power is supplied by the USB-mini connector and lets the user know the sensor hub is active. The wiring for the power LED is from and GND point to the 3.3V out on physical pin 1.

3.5 RTL_433 Radio Dongle

The following shows the configuration of a Sensor Hub running on a PiZero2W with 64-bit Trixie. You can also see the 433MHz radio interface connected to the USB port.

```
pi@sensorhub2ccf67e4ad4a:~ $ hostnamectl
 Static hostname: sensorhub2ccf67e4ad4a
       Icon name: computer
      Machine ID: 763587e0fc13403da6a51b5c4f860fd9
         Boot ID: a8f38173c87c4ce896d21ab1b2e2bcc1
Operating System: Debian GNU/Linux 13 (trixie)
          Kernel: Linux 6.18.50+rpt-rpi-v8
    Architecture: arm64

pi@sensorhub2ccf67e4ad4a:~ $ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 0bda:2838 Realtek Semiconductor Corp. RTL2838 DVB-T
```

4. Software Architecture

- The RPi Zero 2W runs RaspiOS 64-bit Lite, Debian Trixie. The software (**sensorhub**) is also built and tested for an RPi Zero W 32-bit board running Debian Bookworm Lite. The 32-bit board requires Bookworm because some drivers for the SDR radio are not backward compatible in the 32-bit version of Trixie.
- Code is written in Go and linux bash scripts.
- Local MQTT broker collects sensor data from the open source rtl_433 software.
- Go code subscribes to the MQTT broker for sensor messages and stores the sensor records in a local sqlite3 database.
- A Go thread periodically cleans up the sensor database, removing random sensors that have been collected and never assigned.

- A Go thread runs a REST service on local port 5000 supplying sensor data when requested by an iOS or iPadOS application. It also updates records when requested by the iOS/iPadOS app.
- A separate program (**sensorhubcontrol**) monitors the reset button, watching for a user request to enter configuration mode, forcing the sensor hub to enter local WiFi hotspot mode which allows an iPhone or iPad to connect and configure the sensor server to a local WiFi SSID. Users can also configure a sensor hub using a browser on a computer on the same network by entering **http://10.42.0.1:5000/** to access the configuration page directly on the Sensor Hub.

4.1 Two separate, communicating programs

There are two independent go applications that implement the Sensor Hub services. One is the **sensorhubcontrol** program. It's function is to control the hardware power and status LEDs as well as watch for rest button presses. **sensorhubcontrol** monitors a configuration file **/var/sensorhub/sensorhub.conf** and a LED status file **/var/sensorhub/led.state**. If the reset button is pressed for 5 seconds, sensorhubcontrol deletes the **sensorhub.conf** file causing **sensorhub** program to enter configuration mode and start the hotspot. **sensorhubcontrol** also watches the led.state file and sets the status LED based on the string value present in the file. The **sensorhub** program can cause the status light to change by updating the value in this file. Keeping the two programs separate facilitates real-time control of the Sensor Hub hardware while maintaining an independent sensor hub function.

4.2 Use of Bonjour services

sensorhub advertises its availablity using a generally available service called Bonjour. Bonjour is Apple's implementation of zero-configuration networking. Apple has made the source code of the Bonjour multicast DNS responder, the core component of service discovery, available as a Darwin open source project. The project provides source code to build the responder daemon for a wide range of platforms, including Mac OS 9, macOS, Linux, *BSD, Solaris, VxWorks, and Windows. The Bonjor service is included in Raspberry Pi OS by **avahi-daemon**.

Bonjour locates devices such as printers, other computers, and the services that those devices offer on a local network using multicast Domain Name System (mDNS) service records. This means that a Sensor Hub can be readily discovered on a LAN without the client polling every device on the network. Using Bonjour also makes implementing a client on Apple iOS practical because iOS typically does not permit a dvice (iPhone) to poll all network devices. The iOS client just needs to look for a Bonjour device advertising as **_sensorhub._tcp**. The avahi daemon running on Raspberry Pi advertises the Sensor Hub.

4.3 REST API

All data collected by the Sensor Hub's radio is processed in real-time. Validated sensor data is inserted (upserted) into a sqlite3 database stored at **\var\sensorhub\weather.db**. Each sensor managed by the Sensor Hub has its own record in the database. As sensor values are updated, **sensorhub** program replaces the latest values for the record in the database. Only one record per sensor is maintained.

Clients request lists of Sensor Hubs and Sensors using a simple REST API. Clients can also manage the Sensor records by renaming them, adding location details, and including descriptions.

The REST API can be invoked from the command line using **curl**, from a Browser using a url, and from an Apple iOS client (which internally uses the REST API). API endpoints are available for lists of Sensors. Endpoints are available for POSTing updates to Sensor records. The sqlite3 database stores Sensor records using the unique hostname assigned to each Sensor Hub, e.g. sensorhub88a29e330a21. Multiple Sensor Hubs can be present on a LAN. Each Hub has its own copy of the Sensor records identified by its hostname.

4.4 Reverse proxy service for enabling remote access from anywhere

The basic **sensorhub** application connects the Hub to a local network via WiFi. To access the sensors it collects, your must be on a computer or other device connected to the same WiFi. To gain access to the Hub remotely outside the local WiFi, the Hub must be visible to DNS across the Internet.

There are multiple ways to make this happen. One is to get a dedicated IP assigned to the Hub. That requires having a domain name and then assigning an IP address to the Hub. Another way is to use a VPN service and access the Hub through the VPN. A third way is to use a low-cost reverse proxy service that creates a virtual pipe between the Hub and the service provider. You connect your computer or device to the service provider through a dedicated https session. The service routes communications to and from the Hub through it's virtual tunnel. The reverse proxy is usually the cheapest and easiest way to gain remote access without hosting your own domain and server or paying for an expensive VPN.

sensorhub has been tested with a reverse proxy service from **pinggy.io**. Cost is a reasonable $30 per year at the time of this writing (2026). The necessary scripts are included in this repository along with step-by-step instructions for configure the reverse proxy in the system design specification PDF.

4.5 [Optional] REST API transport modes security - Default is open

For compatibility, the REST API continues to use unauthenticated HTTP on port 5000 when no `REST_TLS_CERT`, `REST_TLS_KEY`, or `REST_BEARER_TOKEN` variables are set. This legacy mode does not encrypt sensor data or Wi-Fi passwords and should only be used on a trusted, isolated network.

To enable the built-in HTTPS and bearer-token authentication, configure all three variables for the sensorhub service:

REST_TLS_CERT: path to the TLS certificate PEM file.

REST_TLS_KEY: path to the matching private key PEM file.

REST_BEARER_TOKEN: a random secret of at least 32 characters.

For example, generate a token with openssl rand -hex 32, then configure these values using a root-readable systemd environment file. Keep the private key and environment file readable only by the service administrator. Restart **sensorhub** after configuring them. If only some of the variables are provided, the REST service refuses to start rather than silently falling back to HTTP.

In secure mode, every endpoint requires the token in the Authorization header. Do not put it in a URL or log it. For a private/self-signed certificate, install or provide its CA certificate to the client:

```
curl --cacert sensorhub-ca.crt \
  -H "Authorization: Bearer $REST_BEARER_TOKEN" \
  https://sensorhub.local:5000/sensors
```

Clients must use HTTPS and add the bearer header to both read and write calls. Removing the three variables restores the existing HTTP behavior for clients that have not yet been updated.

We DO NOT USE bearer-token authentication in this implementation because the sensorhub is local to the client. If we use a reverse tunnel like pinggy.io, then the reverse tunnel provides basic API services via https without authentication.

The systemd service script for **sensorhub** hotspot redirects stdout to /var/log/sensorhub.log

4.6 Browser Wi-Fi setup

When **/var/sensorhub/sensorhub.conf** is missing, SensorHub starts its default SensorHub hotspot. Join that network from a phone or computer and open **http://10.42.0.1:5000/** in a browser to scan for networks and configure Wi-Fi. The browser tool uses the same /ssids and /connect-ssid REST endpoints as the iOS client. After a successful setup, reconnect your device to the configured Wi-Fi network.

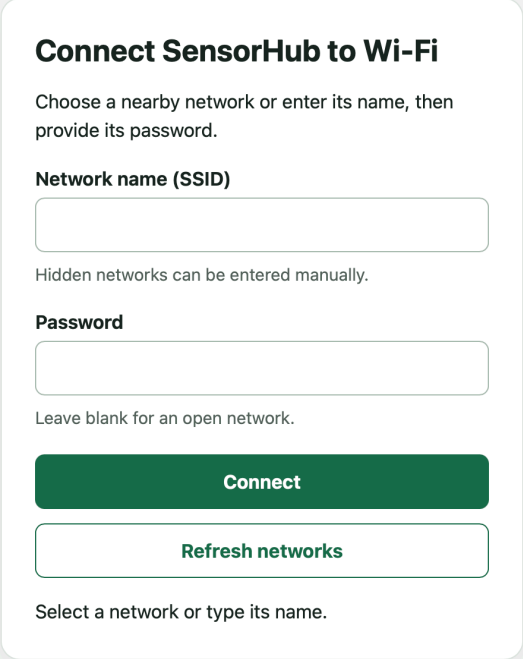
A local WiFi hotspot initially runs on the sensor hub to communicate between the sensor hub and the user's iPhone (or browser via REST API) to enable Sensor Hub configuration. The green LED flashes on and off. The iPhone or browser requests the login information for the WiFi network you want to connect to. **sensorhub** then configures the sensorhub software to connect to the local WiFi network with the user-supplied password. The **sensorhub** renames the sensor hub to "sensorhubxxxxxxxxxxxx", where the x's are the wlan0 MAC address of the Raspberry Pi sensor hub. **sensorhub** restarts avahi-daemon using the new hostname, and when the sensor hub is active again, it connects to the local WiFi using "sensorhubxxxxxxxxxxxx" and the user-supplied WiFi password.

Optionally, the Sensor Hub provides a means to configure a new Hub using only a local computer and web browser. Open the browser and enter the following URL, then follow the prompts:

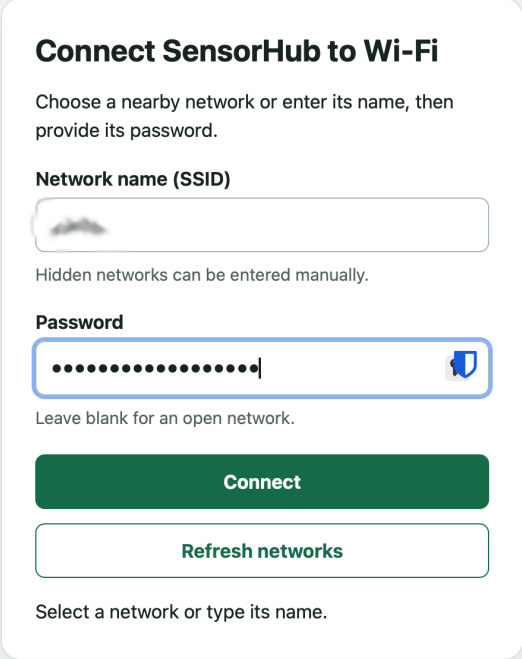
```
    http://10.42.0.1:5000/
```

If the user presses and holds the Reset button for 5 seconds, the sensor hub process **sensorhubcontrol** will initiate a reconfiguration and start the broadcast over as a standalone WiFi server waiting for a connection from an IOS app (or a browser at the URL above) and setup to new SSID.

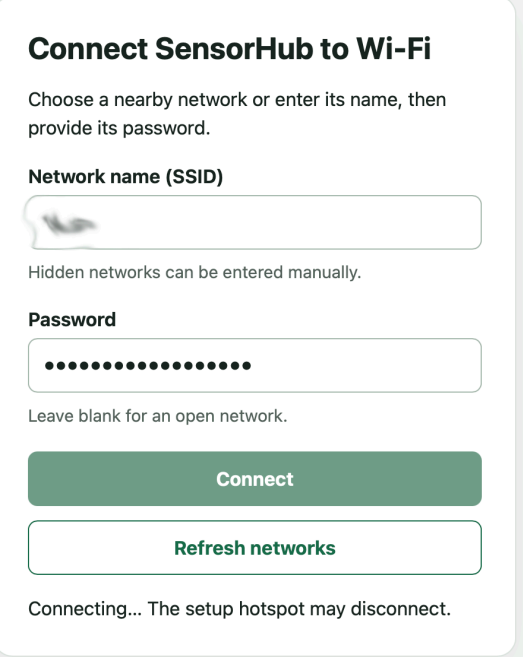
The following screenshots show the browser-based WiFi setup.



Browser - Select a target network



Browser - Enter password for network



Browser - sensorhub connects

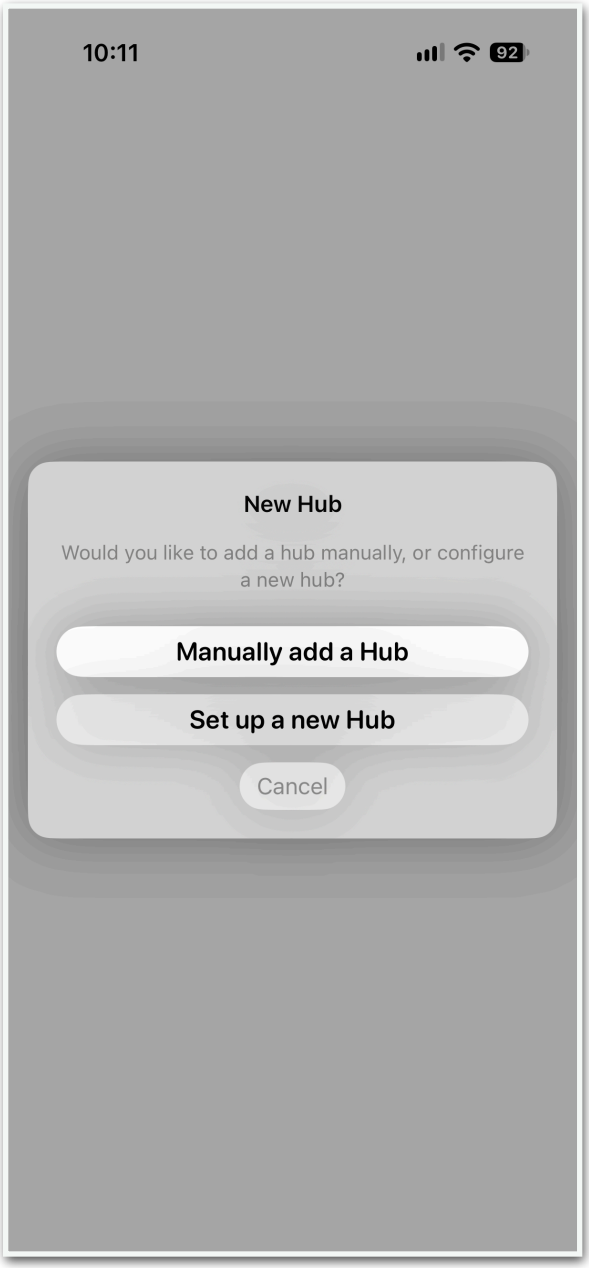
The sensor hub also supports remote access using a reverse proxy service. Currently the only service tested is pinggy.io. Future updates may incorporate other services. Once configured, the reverse proxy service replaces the local access for the sensor hub, and all REST API responses are funneled through the reverse proxy channel. The user can turn the reverse proxy service on and off by using the button on the Sensor Hub edit page.

4.7 iPhone Wi-Fi setup (requires iOS developer and XCode tools)

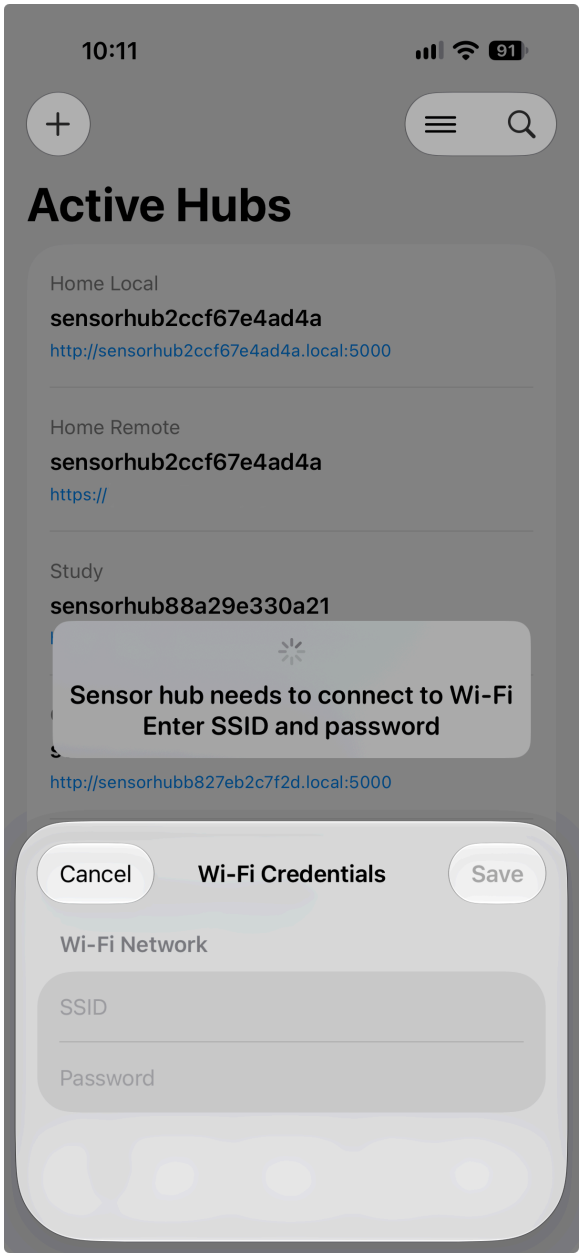
The iOS application is not required for the functioning of the Sensor Hub. Everything on the Sensor Hub can be accessed via command-line or browser. For example, you can use the REST API service to supply data to a Node-Red application to format and display the sensor information.

A sibling open source project on github https://github.com/cjr29/sensorhubXCode provides an Apple iOS application that enables a rich environment for displaying the sensor data. It also supports management of multiple sensor hubs. Remote hubs are easily added.

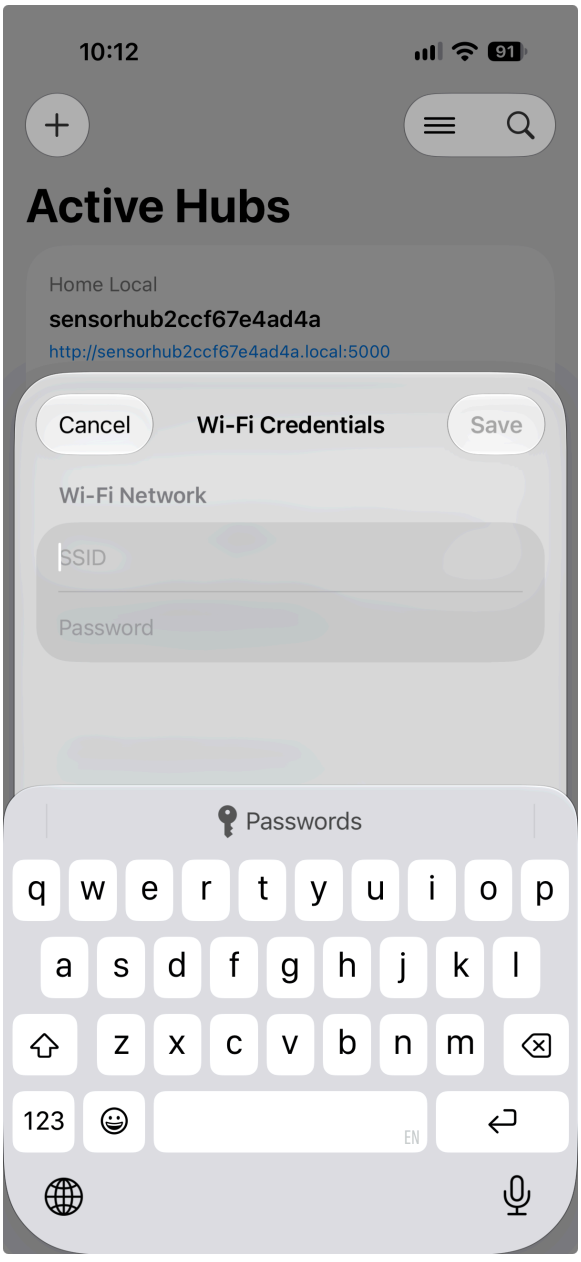
The following images are the iPhone sequence when setting up a new Hub:



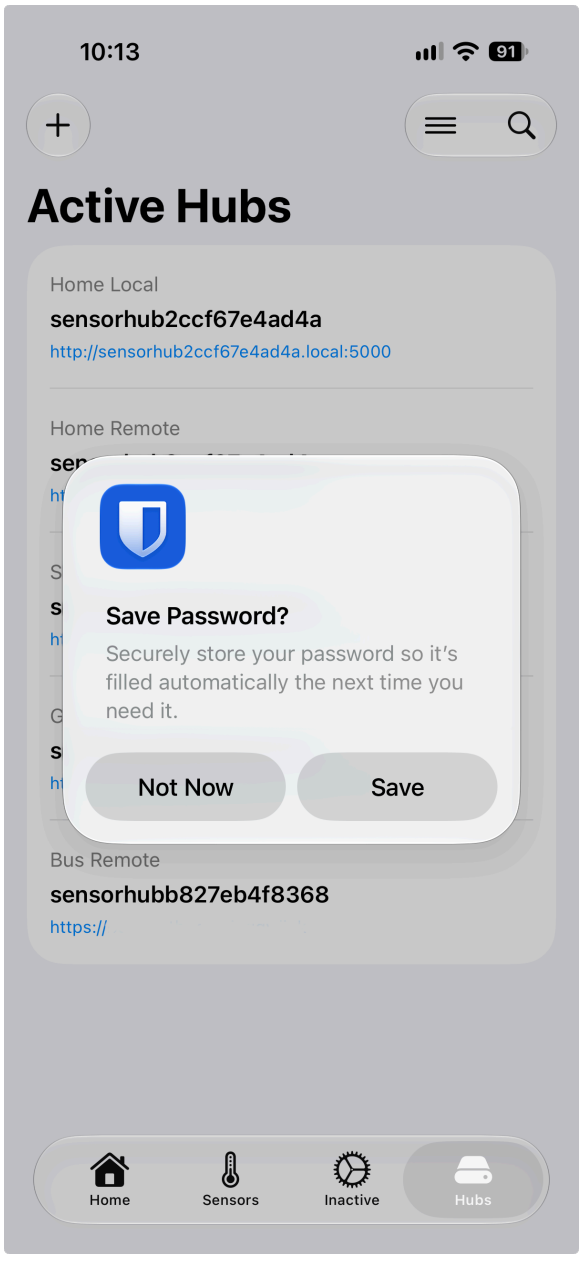
1. Select: Set up a new Hub



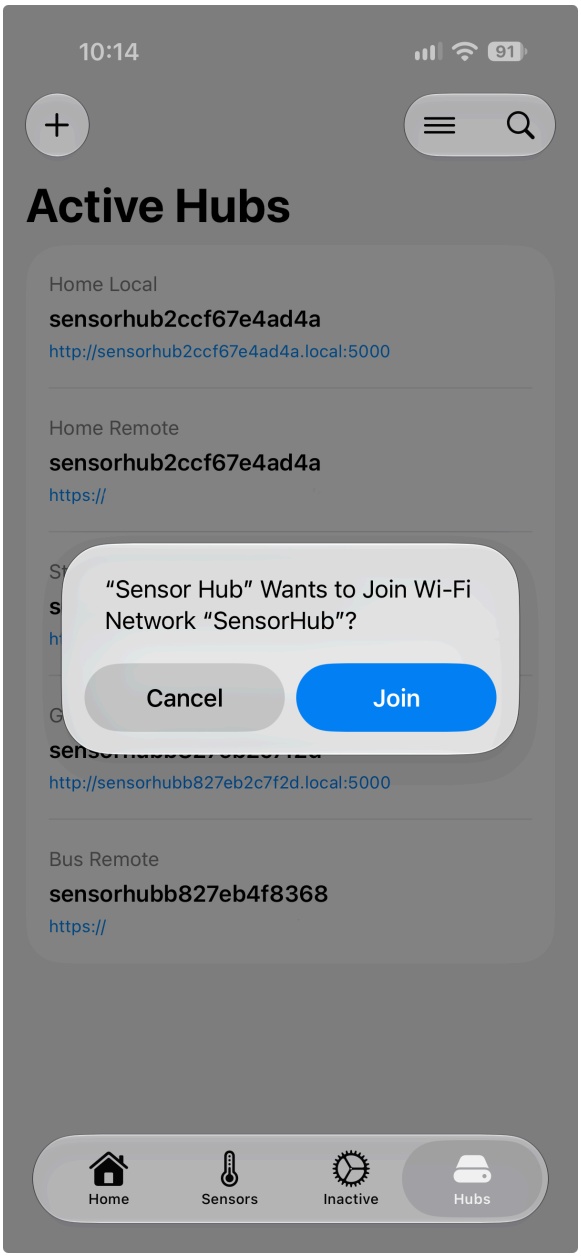
2. Enter selected WiFi login info



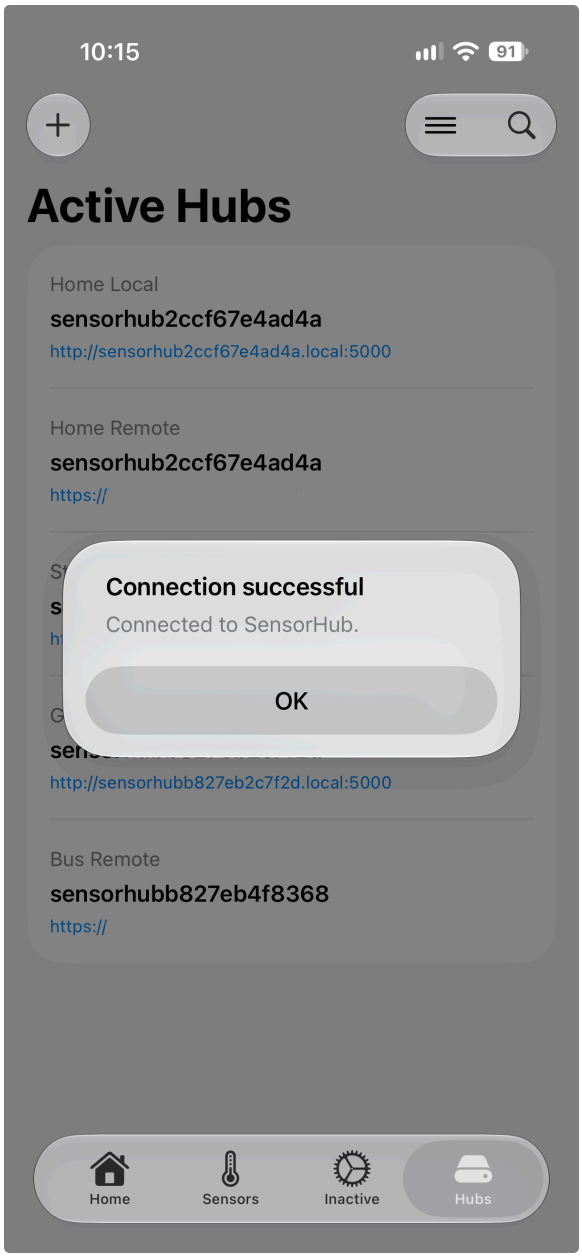
3. WiFi credentials may be saved in Passwords. If so, select from there.



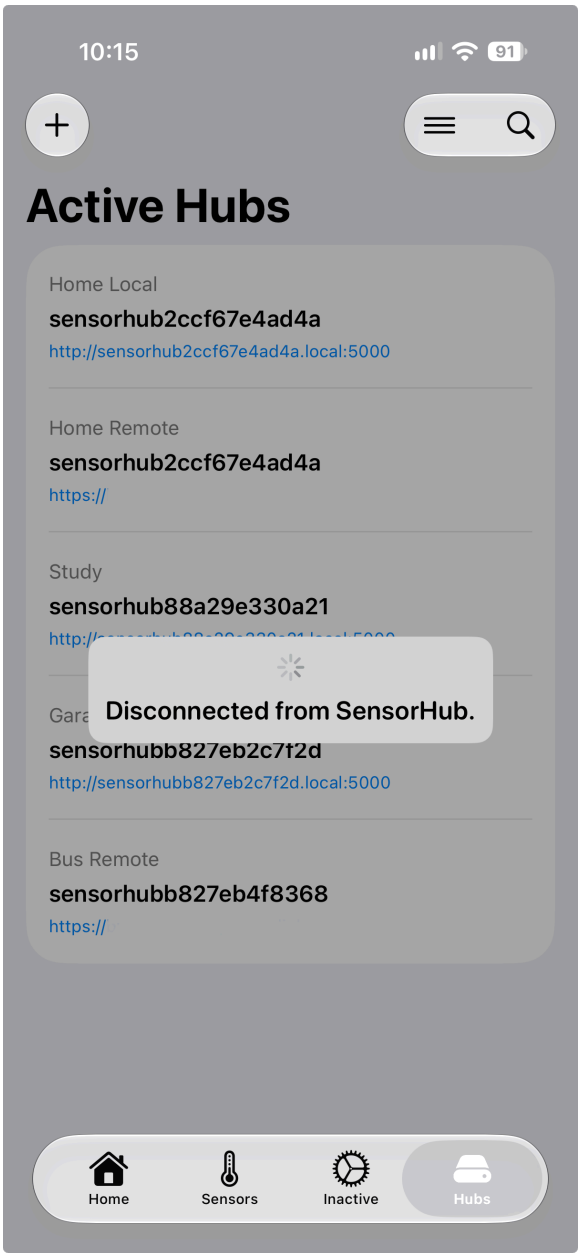
4. You can save for future use in your Password keeper



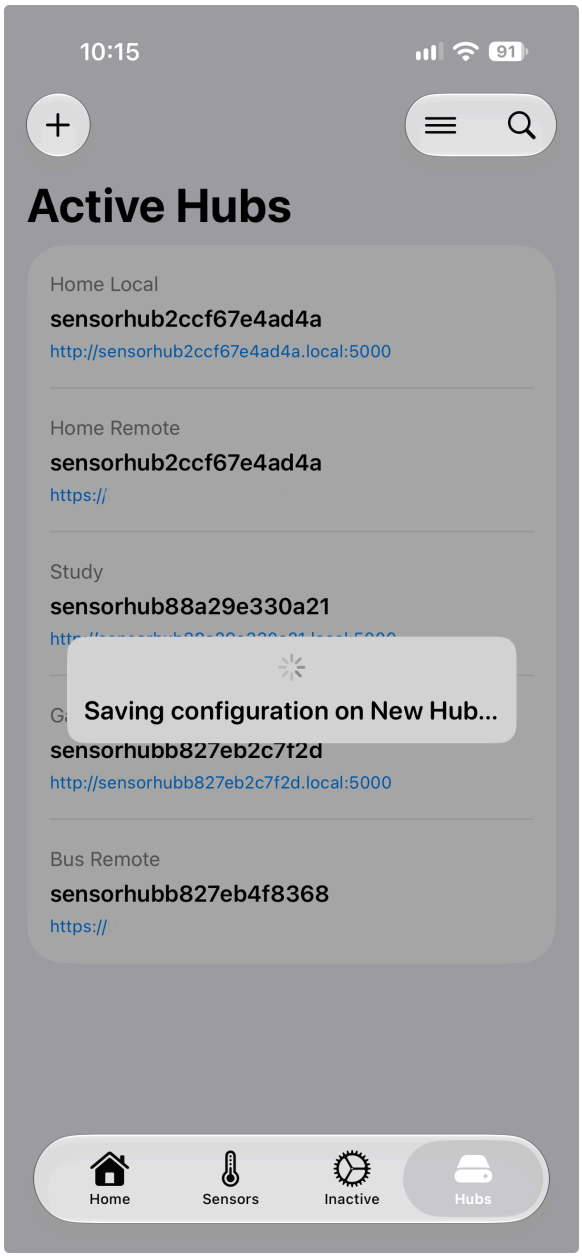
5. Allow new Sensor Hub to join your WiFi



6. Hub was able to connect to WiFi



7. App disconnects from Hotspot



8. Sensor Hub finishes its setup

4.9 rtl_433 (open source)

The sensor hub runs the rtl_433 driver and service searching for signals from temperature and humidity sensors transmitting on 433 MHz. Each message it receives from a sensor is processed and passed to the local MQTT Broker as a topic named 'weather/sensors'. The message payload is formatted in JSON supplied by the sensor.

NOTE: *The radio dongle is connected to the Pi via a USB-A to USB-micro adapter. Be sure that that adapter can handle full data signals. Some cables are unreliable and cause the radio to drop its connection with the Pi after several hours. The cause is unknown, but testing shows the particular extender is critical to reliable functioning of the RTL radio.*

5.0 Software Design and Operation

This section discusses the data structures and algorithms used by the **sensorhub and sensorhubcontrol** Go programs running on the Pi.

5.1 Data Structures

5.1.1 SENSORS Table

The primary persistent data structure used by the Sensor Hub system is the sqlite3 database maintained on the Raspberry Pi host. This database consists of a single table schema (SENSORS) containing a unique record for each SENSOR that the Pi has detected using the RTL 433 MHz radio receiver attached to its USB port.

The following table shows the schema for SENSORS:

Field	Data Type	Description	Example Value
ID	INTEGER	PRIMARY KEY assigned automatically by SQLITE and used for indexing and updating	151
TIME	TEXT	Sensors provide a text-based date/time.	"2026-09-21 02:18:19"
SENSORID	INTEGER	ID returned with the payload from a sensor	15122
MODEL	TEXT	model name returned in the sensor payload	"Acurite-Tower"
CHANNEL	TEXT	Some sensors provide a configuration choice of channel A, B, or C. This can also appear as an integer value. The database stores it as TEXT to simplify JSON processing.	"A"
BATTERY	INTEGER	Some sensors provide a battery level/ health indicator. Most are either 0 or 1. It is stored in the database, but not currently used.	1
TEMPERATURE	NUMERIC	Temperature is supplied by sensors in various formats, and it is converted into a decimal-based value for consumption by JSON formatters.	76.64
HUMIDITY	NUMERIC	Humidty is supplied by sensors in various formats, and it is converted into a decimal-based value for consumption by JSON formatters.	47
SENSORNAME	TEXT	User field. Applications can update this with a text value through the REST API.	“Utility Bay”
STATION	TEXT	Computed text field comprised of ‘sensorhub’ concatenated with the 12-character MAC address of the Pi wlan0 interface. Serves to identify the sensor hub that provided the data record when sent via JSON tthrough the REST API.	“sensorhubb827eb4f8368 ”
LOCATION	TEXT	User field. Applications can update this with a text value through the REST API.	“ ”
DESCRIPTION	TEXT	User field. Applications can update this with a text value through the REST API.	“ ”
ACTIVE	INTEGER	User controlled field via REST API. 0 = INACTIVE, 1 = ACTIVE	1

An example JSON record for above database entry is shown below:

```
{"ACTIVE":1,"BATTERY":1,"CHANNEL":"A","DESCRIPTION":"
","HUMIDITY":47,"ID":151,"LOCATION":" ","MODEL":"Acurite-
Tower","SENSORID":15122,"SENSORNAME":"Utility
Bay","STATION":"sensorhubb827eb4f8368","TEMPERATURE":76.64,"TIME":"2026-09-21
02:18:19"}
```


5.2 Hub and App Control

Discuss the reset button and what it does to **/var/sensorhub/sensorhub.conf**. Also explain that there is a local browser-based configuration page for the hotspot at http://10.42.0.1:5000 that can be used to configure (or reconfigure) a sensorhub. Also available from the iOS app.

5.3 Interfaces and Thread Signaling

The go language sensorhub application uses goroutines and signals to communicate between components of the program.

5.3.1 processsensordata (Goroutine)

This is a Go subroutine that runs constantly. It subscribes to all 'weather/sensors' topics on the MQTT broker. As a message and payload is processed, the local sqlite3 database is checked to see if the sensor has been entered already. If not, a new record is created. If it already exists, the sensor record is updated with the new temperature, humidity, and time.

5.3.2 restservice (Goroutine)

This subroutine provides a REST API service on port 5000 to external clients giving access to the sensor data as a JSON array. Clients can request a list of sensors, a specific sensor, delete a sensor, and PATCH (update) a sensor. These REST API calls can be made by an application, a browser, or command line using curl. Response data is returned as JSON. By default, the API is unsecured since we only run on the local network. If you use a reverse proxy service for remote access, the proxy provides its own security and access control depending on how you configure it.

List of API calls (Go code):

```
    r.Get("/", handleWiFiSetupPage)
    r.Get("/sensors", handleGetSensors)
    r.Get("/sensor/{id}", handleGetSensorID)
    r.Get("/station", handleGetStation)
    r.Get("/ssids", getSSIDs)
    r.Post("/connect-ssid", handlePostConnectSSID)
    r.Patch("/sensor/rename", handlePatchSensorRename)
    r.Patch("/sensor/full", handlePatchSensorFull)
    r.Patch("/hostname", handleHostRename)
```

5.3.3 expiredsensors (Goroutine)

This subroutine runs periodically to check the weather.db for sensors that are INACTIVE and have been in the database for more than a day. If it finds any, the records are deleted to clean up the database from sensors that are added inadvertently by the sensor hub. For example, an RV parks next to the location of the sensor hub and the RV has a temperature sensor broadcasting. It will be ignored by the user and must be removed from the database to avoid useless clutter.

5.3.4 monitorwifi (Goroutine)

Runs as a background thread. Periodically checks availablity of the wifi interface and the Internet. Sets the LED FLASHGREEN status in the led.state file to tell the sensorhubmonitor program to flash the status LED.

5.3.5 writeledstate (Goroutine)

Runs in background thread waiting for a signal from the main program to change the status of the led.state file.

6.0 Sensor Hub Software Build

6.1 Update Raspberry Pi OS

Using Raspberry Pi Imager, create a 32GB microSD card with 64-bit Trixie Lite (for Pi 2W) or 32-bit Bookworm Lite (for Pi W) and install. Include rpi-connect, ssh, and preinstalled keys. If you plan to log into the Pi and manually configure it, be sure to enable SSH and provide the SSH key in Imager.

Note: We use Bookworm on the 32-bit Pi because of issues with SDR drivers in Trixie. The Trixie 64-bit drivers are fine for 64-bit Pi.

```
$ sudo apt update
$ sudo apt upgrade
```

6.2 Install git and sensorhub, sensorhubcontrol repositories

```
$ cd ~
$ sudo apt install git
$ git config --global user.email "<your email address>"
$ git config --global user.name "<your name>"
$ git clone git@github.com:cjr29/sensorhub.git
$ git clone git@github.com:cjr29/hubcontrol.git,
$ cd sensorhub
```

6.3 Install Mosquitto

```
$ sudo apt update
$ sudo apt install -y mosquitto mosquitto-clients
# Enable mosquitto restart on boot
$ sudo systemctl enable mosquitto
# Start mosquitto now
$ sudo systemctl start mosquitto
```

6.4 Test Mosquitto

```
$ mosquitto_sub -h localhost -p 1883 -t sensorhub/weather/sensors
```

6.5 Create and Configure Log Files and working directory

/var/lib/sensorhub

```
$ cd ~
$ sudo mkdir /var/sensorhub
$ sudo chmod 775 /var/sensorhub
$ sudo chown root:root /var/sensorhub
# Create systemd and logging files
$ sudo mkdir -p /var/lib/sensorhub /var/log/sensorhub
$ sudo chown -R root:root /var/lib/sensorhub /var/log/sensorhub
# sensorhub uses /var/sensorhub/sensorhub.conf to determine if the WiFi has
already been configured
# sensorhub.conf contains the SSID of the configured network
```

6.6 Create logrotate configuration file

Enter the following script, and save as **/etc/logrotate.d/sensorhub** where,

- new log monthly
- rotate logs every 6 months
- once full, zip them
- after rotation, send SIGHUP to let sensorhub program know the log file has changed so it can open the new file

```
var/log/sensorhub/sensorhub.log {
    monthly
    rotate 6
    compress
    delaycompress
    missingok
    notifempty
    # create 0644 sensorhubuser sensorhubuser
    sharedscripts
    postrotate
        # send SIGHUP to the service to make it reopen logs after rotation
        if systemctl is-active --quiet sensorhub.service; then \
            systemctl kill --signal=SIGHUP sensorhub.service;
        fi;
    endscript
}
```

6.7 Install sqlite3

```
$  sudo apt install sqlite3
```

6.8 Configure Bonjour advertising

Edit avahi-daemon config file to turn off advertising on _workstation._tcp.

```
$ sudo nano /etc/avahi/avahi-daemon.conf
```

```
# Set workstation publishing off (add or change):
publish-workstation=no
# Restart avahi-daemon
$ sudo systemctl restart avahi-daemon
```

Bonjour is used to let the network know you are providing a service on port 5000 that lets apps find sensorhubs.

6.9 Add sensorhub.service:

Create **/etc/avahi/services/** and add the following script to it:

```
$ sudo nano /etc/avahi/services/sensorhub.service
<?xml version="1.0" standalone='no'?>
<!DOCTYPE service-group SYSTEM 'avahi-service.dtd'>
<service-group>
  <name replace-wildcards="yes">%h</name>
  <service>
    <type>_sensorhub._tcp</type>
    <port>5000</port>
    <txt-record>device=Sensor Hub</txt-record>
  </service>
</service-group>

# If added after booting, must do a systemctl daemon-reload, systemctl
restart avahi-daemon

$ sudo nano /etc/avahi/avahi-daemon.conf
# Change the following line in avahi-daemon.conf
publish-workstation=no
# Save and
$ sudo systemctl restart avahi-daemon
```

6.10 Configure profile for local hotspot

The important step in creating the sensorhub image is to remove the default netplan-wlan0-pluto connection profile created by raspi-config (or Raspberry Pi Imager). It causes problems with the program using the SensorHubHotspot connection to connect to a local wifi. The only wlan0 connection profile that should exist is wifi-home.

6.10.1 Delete that connection:

```
$ sudo nmcli con delete netplan-wlan0-pluto
```

Submit the following commands once for the sensorhub. It will set up a hotspot profile for the sensor hub that can be invoked programmatically whenever it is necessary to connect to wifi. The hotspot profile ‘SensorHubHotspot’ allows devices to connect to it (default IP is 10.54.0.1) using the ssid ‘SensorHub’ and password ‘ConfigureMe123’.

```
# Create a WPA-PSK hotspot connection
$ sudo nmcli connection add type wifi ifname wlan0 con-name SensorHubHotspot
autoconnect no ssid SensorHub
# Configure it as an access point with WPA-PSK
nmcli connection modify SensorHubHotspot \
  802-11-wireless.mode ap \
  802-11-wireless.band bg \
  ipv4.method shared \
  ipv6.method ignore \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk 'ConfigureMe123'
Other commands available:
# Start the hotspot
$ sudo nmcli connection up SensorHubHotspot

# Turn Off: Shut down the broadcast safely.
$ sudo nmcli connection down SensorHubHotspot

# Check status of hotspot
$ nmcli connection show
# Check status of service
$ sudo journalctl -u sensorhub.service

# Generate a Join QR Code: Easily connect smartphones or tablets by
outputting a QR code directly inside your terminal
$ nmcli device wifi SensorHubHotSpot show

# Delete the hotspot profile
$ sudo nmcli connection delete "SensorHubHotspot"
```

6.11 Install rtl_433 radio software (to /usr/bin)

```
$ sudo apt-get install rtl-433
# Reboot
# Test radio code
$ /usr/bin/rtl_433 -v -R 11 -R 40 -R 55 -R 163 -C customary
# Kill the process once validated. We will start the service using systemd on
reboot.
```

Note: *On some Raspberry PI boards, the sensorhubcontrol.service program may grab the USB bus and not release it, causing rtl-temp-sensor.service to fail. To resolve this issue, stop and disable the sensorhubcontrol.service and reboot the CPU.*

Sometimes, the OS loses contact with the USB radio dongle. To check if it is available:

```
pi@sensorhub2ccf67e4ad4a:~ $ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 002: ID 0bda:2838 Realtek Semiconductor Corp. RTL2838 DVB-T
```

If Device 002 RTL2838 DVB-T disappears, the OS can no longer control it. The only resolution if the device becomes invisible is to power cycle the USB device. Unplug it and let it restart.

6.12 Configure temp-sensor-rtl service to systemd from repository directory

```
# From ~/sensorhub, copy the sensor service
$ cd ~/sensorhub
$ sudo cp temp-sensor-rtl.service /lib/systemd/system
# Start the service
$ sudo systemctl start temp-sensor-rtl
$ sudo systemctl enable temp-sensor-rtl
# Test that temp-sensor-rtl is publishing to mosquitto
$ mosquitto_sub -h localhost -t "sensorhub/#"
# See logs from temp-sensor-rtl.service
$ sudo journalctl -u sensorhub-hotspot.service
```

To check if the sensor service is running:

```
$ sudo systemctl status temp-sensor-rtl
```

Result should be:

```
@sensorhub2ccf67e4ad4a:~/sensorhub $ sudo systemctl status temp-sensor-
rtl.service
temp-sensor-rtl.service - RTL_433 Software Defined Radio service
     Loaded: loaded (/usr/lib/systemd/system/temp-sensor-rtl.service;
enabled; preset: enabled)
     Active: active (running) since Mon 2026-08-03 12:53:15 EDT; 1 day 8h ago
 Invocation: 8198078836984b87ad3ce8d84e5398f3
   Main PID: 2521 (rtl_433)
      Tasks: 3 (limit: 218)
        CPU: 1h 45min 56.647s
     CGroup: /system.slice/temp-sensor-rtl.service
             └─2521 /usr/bin/rtl_433 -R 8 -R 11 -R 12 -R 20 -R 40 -R 54 -R 55
-R 73 -R 75 -R 76 -R 91 -R 112 -R 113 -R 163 -v -C customary -F mqtt://
localhost 1883 events >

Aug 03 12:53:15 sensorhub2ccf67e4ad4a systemd[1]: Started temp-sensor-
rtl.service - RTL_433 Software Defined Radio service.
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: rtl_433 version 25.02
(2025-02-19) inputs file rtl_tcp RTL-SDR SoapySDR
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: MQTT: Publishing MQTT
data to localhost port 1883
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: MQTT: Publishing events
info to MQTT topic "sensorhub/weather/sensors".
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: Use "-F log" if you want
any messages, warnings, and errors in the console.
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: Detached kernel driver
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: Found Rafael Micro R820T
tuner
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: Exact sample rate is:
250000.000414 Hz
Aug 03 12:53:16 sensorhub2ccf67e4ad4a rtl_433[2521]: [R82XX] PLL not locked!
```

6.13 Copy sensorhub and hubcontrol files

```
# Copy sensorhub and hubcontrol files
$ cd ~/sensorhub
$ sudo mkdir /usr/local/bin/sensorhub
$ sudo cp sensorhub /usr/local/bin/sensorhub/
$ sudo cp sensorhub.service /lib/systemd/system/
$ sudo cp temp-sensor-rtl.service /lib/systemd/system/
$ sudo cp -r static /usr/local/bin/sensorhub/
$ sudo cp -r templates /usr/local/bin/sensorhub/
# Note that in the following copy, sensorhub.service.avahi is renamed to
sensorhub.service
$ sudo cp sensorhub.service.avahi /etc/avahi/services/sensorhub.service
$ cd ~/hubcontrol
$ sudo cp hubcontrol /usr/local/bin/
$ sudo cp sensorhubcontrol.service /lib/systemd/system/
```

6.14 Enable systemd services

```
# Enable systemd services
$ sudo systemctl enable temp-sensor-rtl
$ sudo systemctl enable sensorhub.service
$ sudo systemctl enable sensorhubcontrol.service
```

6.15 Configure profile for local hotspot

An important step in creating the sensorhub image is to remove the default netplan-wlan0-pluto connection profile created by raspi-config (or Raspberry Pi Imager). It causes problems with the program using the SensorHubHotspot connection to connect to a local wifi.

Delete that connection:

$ sudo nmcli con delete netplan-wlan0-pluto

The only wlan0 connection profile that should exist is wifi-home

Submit the following commands once for the sensorhub. It will set up a hotspot profile for the sensor hub that can be invoked programmatically whenever it is necessary to connect to wifi. The hotspot **profile** 'SensorHubHotspot' allows devices to connect to it (default IP is 10.54.0.1) using the **ssid** 'SensorHub' and password 'ConfigureMe123'.

```
# Create a WPA-PSK hotspot connection
$ sudo nmcli connection add type wifi ifname wlan0 con-name SensorHubHotspot
autoconnect no ssid SensorHub
# Configure it as an access point with WPA-PSK
nmcli connection modify SensorHubHotspot \
  802-11-wireless.mode ap \
  802-11-wireless.band bg \
  ipv4.method shared \
  ipv6.method ignore \
  wifi-sec.key-mgmt wpa-psk \
  wifi-sec.psk 'ConfigureMe123'
```

Other commands available:

```
# Start the hotspot
$ sudo nmcli connection up SensorHubHotspot

# Turn Off: Shut down the broadcast safely.
$ sudo nmcli connection down SensorHubHotspot

# Check status of hotspot
$ nmcli connection show
# Check status of service
$ sudo journalctl -u sensorhub.service

# Generate a Join QR Code: Easily connect smartphones or tablets by
outputting a QR code directly inside your terminal
$ nmcli device wifi SensorHubHotSpot show

# Delete the hotspot profile
$ sudo nmcli connection delete "SensorHubHotspot"
```

10.0 Copy executables and templates to /usr/local/bin/sensorhub

```
$ sudo mkdir /usr/local/bin/
$ sudo cp sensorhub /usr/local/bin/
```

11.0 Start sensorhub Service Under systemd

sensorhub provides a standalone WiFi server to manage the sensor hub initial configuration. It runs under systemd as a service. This program also manages the local HubHotspot whenever WiFi must be configured.

11.1 Step 1 - Create a systemd service file to run sensorhub on boot up

In **/lib/systemd/system**, create a file named **sensorhub.servic**e, and insert the code:

```
[Unit]
Description=Sensor Hub service
After=network.target

[Service]
Type=simple
User=root
Group=root
RuntimeDirectory=sensorhub
PIDFile=/run/sensorhub/sensorhub.pid
WorkingDirectory=/var/lib/sensorhub
ExecStart=/usr/local/bin/sensorhub/sensorhub
Restart=on-failure
RestartSec=5
TimeoutStopSec=15

[Install]
WantedBy=multi-user.target
```

11.2 Step 2 - Check log of running service

```
$ sudo journalctl -u sensorhub-hotspot.service
```

```
$ sudo apt autoremove -y && sudo apt clean
```

12.0 Remote Access via pinggy.io

The basic installation of sensorhub must be accessed on the same WiFi network as the iOS device using it. The iOS app uses the Sensor Hub url to retrieve sensor data. To access the sensor hub data remotely outside the local WiFi, a reverse tunnel service is needed. The following instructions tell how to set up such a service using pinggy.io. There are other reverse tunnel services, but Sensor Hub works well with pinggy.io, and pinggy.io is simple to subscribe to and set up. The Pro plan for a single tunnel is only $30 per year, making it a reasonable source for remote access.

12.1 Step 1: Go to the pinggy.io subscription

Go to the pinggy.io web site and subscribe to the Pro plan for one tunnel at $30/yr.

12.2 Step 2: Configure the new tunnel

On the pinggy.io dashboard page, go to **Configure Tunnel**.

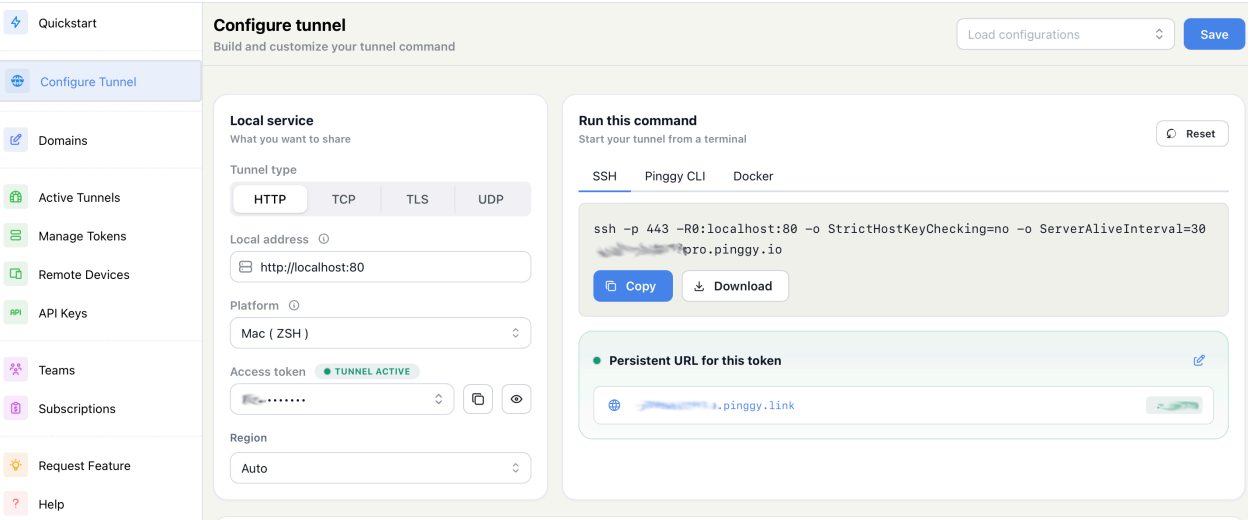
- Under Persistent URL for this token, edit the subdomain name from the default provided to '<your link>.pinggy.link'
- Under **Local Service** fill in the fields:

  Local address: http://sensorhubb827eb4f8368.local:5000

  Platform: Linux (Bash)

  Access token: select the proper token for this tunnel

  Region: Auto



12.3 Step 3: Save the configuration

12.4 Step 4: Copy the SSH command block

```
ssh -p 443 -R0:sensorhubb827eb4f8368.local:5000 -o StrictHostKeyChecking=no
-o ServerAliveInterval=30 <your key>@pro.pinggy.io
```

12.5 Step 5: Create pinggy_tunnel.sh script

```
$ sudo nano /usr/local/bin/pinggy_tunnel.sh

# Paste in this script the copied SSH command block
ssh -p 443 -R0:sensorhubb827eb4f8368.local:5000 -o StrictHostKeyChecking=no
-o ServerAliveInterval=30 <your key>@pro.pinggy.io
# Make executable
$ sudo chmod +x /usr/local/bin/sensorhub/pinggy_tunnel.sh
```

12.6 Step 6: Create a Systemd Service

Create a systemd service file /lib/systemd/system/pinggy-tunnel.service:

```
[Unit]
Description=Pinggy Tunnel Startup

[Service]
ExecStart=/usr/local/bin/pinggy_tunnel.sh
Restart=on-failure
RestartSec=10s
User=root

[Install]
WantedBy=multi-user.target
```

12.7 Step 7: Reload Systemd and enable the service

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable pinggy-tunnel.service
# Run initially to allow access
pinggy_tunnel.sh
# Once allowed, it can run in background
```

12.8 Step 8: Access the REST service like this

```
# For remote, in browser type:
https://<your hostname>.-.pinggy.link/sensors

# For local network, in browser type:
http://sensorhubb827eb4f8368.local:5000/sensors
```

12.9 Troubleshooting

- Sometimes when the log or journal files are rotated, sensorhub loses the pinggy tunnel connection. If restarting or rebooting the program or hub does not restore the tunnel, go to the pinggy.io dashboard and manually terminate the tunnel to the sensorhub. Within a minute the hub will restart the tunnel itself and restore the connection.

13.0 systemd commands

The following provides the latest 50 lines of the systemd journal for the sensorhub program:

```
# View full logs for the service (use -f to "follow" real-time logs)
$ journalctl -u sensorhub.service --no-pager -n 50  # -n 50 shows last 50
lines
```

14.0 Log commands

The following will display a continuous scrolling screen of the latest entries in the sensorhub.log:

```
$ tail -f -n 30 /var/log/sensorhub/sensorhub.log
```

15.0 Viewing sqlite3 database

Log into the sensorhub using whichever method you configured with Raspberry Pi Imager and go to the directory where the weather data is stored. Open the database and inspect the data:

```
$ cd /var/sensorhub
$ sqlite3 weather.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> select * from SENSORS;
167|2026-10-23 10:59:27|14930|Acurite-Tower|A|1|55.22|64|Porch|
sensorhub88a29e330a21| | |1
168|2026-10-23 10:59:21|16074|Acurite-Tower|C|1|63.32|43|Garage|
sensorhub88a29e330a21| | |1
169|2026-10-23 10:59:27|1997|Acurite-5n1|A|1|56.8|80|Deck|
sensorhub88a29e330a21| | |1
170|2026-10-23 10:59:22|6914|Acurite-Tower|A|1|72.32|46|Back BR|
sensorhub88a29e330a21| | |1
171|2026-10-23 10:59:16|6|Acurite-606TX|1|1|30.92|-99|Bus Fridge|
sensorhub88a29e330a21| | |1
172|2026-10-23 10:59:24|3036|Acurite-Tower|B|1|53.96|65|Bar Wine Fridge|
sensorhub88a29e330a21| | |1
173|2026-10-23 10:58:53|16|AmbientWeather-TX8300|0|-1|75.02|-99|Study|
sensorhub88a29e330a21| | |1
174|2026-10-23 10:59:31|146|LaCrosse-TX141Bv3|1|1|57.56|-99|MBR Projection
Deck|sensorhub88a29e330a21| | |1
175|2026-10-23 10:59:31|28|LaCrosse-TX141Bv3|1|1|56.84|-99|MBR Wall|
sensorhub88a29e330a21| | |1
sqlite> .quit
$
```

16.0 Testing REST Service from another terminal

The following are examples of curl commands used to test the REST API.

```
# Retrieve the first sensor record
$ curl -i http://sensorhub88a29e330a21:5000/sensor/1

# Also accessible using avahi naming:
$ curl -i http://sensorhub88a29e330a21.local:5000/sensor/1

# Rename ID 3
$ curl -X PATCH -H "Content-Type: application/json" -d '{"ID":3,
"SENSORNAME":"Front Porch - renamed"}' http://
sensorhub88a29e330a21.local:5000/sensor/rename
$
$ {"SENSORNAME":"Front Porch - renamed","message":"Sensor 3 updated"}

# Retrieve all sensor records from the station
curl -i http://sensorhub88a29e330a21.local:5000/sensors

# Retrieve the station (hub) info
# hostname (mDNS name)
curl -i http://sensorhub88a29e330a21.local:5000/station
HTTP/1.1 200 OK
Content-Type: application/json
Date: Sat, 15 Aug 2026 12:52:53 GMT
Content-Length: 36

{"STATION":"sensorhub88a29e330a21"}

Note that whenever the hostname is changed by sensor/rename, the program also
restarts avahi-daemon.service to make the new hostname visible on the local
network. This is done within the hostname.go file.
# Change Hostname
$ curl -X PATCH -H "Content-Type: application/json" -d
'{"HOSTNAME":"sensorhub2"}' http://sensorhub88a29e330a21.local:5000/hostname

# Request to connect to SSID
curl -sS -X POST "http://localhost:5000/connect-ssid" \
  -H "Content-Type: application/json" \
  -d '{"SSID":"MyWiFi","password":"MyPassword"}'
```